PAINTER’S ALGORITHM

Complete Micro Project Study & Viva Guide

Project: 3D Indoor Navigation Using Depth Ordering

Student: Puneeth N **Reg. No.:** 2411021061259

Purpose: Exam revision, project understanding and viva preparation

MASTER IDEA

Painter’s Algorithm is a visibility algorithm that renders polygons from farthest to nearest, so nearer polygons are drawn later and cover farther polygons.

This guide combines the project explanation, poster/PPT concepts, implementation-specific points, limitations, controls and viva-ready answers discussed in this chat.

1. START FROM THE BASICS

What is Computer Graphics?

Computer Graphics is the field of computer science that deals with creating, representing, manipulating and displaying images and visual objects using computers. In this project, computer graphics concepts are used to create and render a 3D indoor environment.

What is the visibility problem?

The visibility problem is the problem of determining which objects or surfaces should be visible to the camera and which objects are hidden behind other objects. For example, if a table is in front of a wall, the table should appear in front of the wall.

Why do we need depth ordering?

Depth ordering is needed to determine which surface is in front of another surface. Without proper depth ordering, a farther object may incorrectly appear over a nearer object.

Why is it called Painter's Algorithm?

It works like a painter painting a picture: the background is painted first and foreground objects are painted later. Later paint covers earlier paint. Similarly, the algorithm draws far objects first and near objects last.

Other names

• Depth-Sorting Algorithm
• Back-to-Front Rendering
• Priority-based polygon rendering approach

2. THE CORE OF YOUR PROJECT

Your project is **3D Indoor Navigation Using Depth Ordering**. The environment contains rooms, a corridor, walls, desks, doors and other furniture. The system must determine which surfaces are closer to the camera and which are farther away.

The four main steps are:

Step	Name	What happens
1	Transform	Convert polygons into camera space.
2	Measure Depth	Calculate distance from camera to polygon centre.
3	Sort	Arrange polygons from farthest to nearest.
4	Draw	Render polygons in the sorted order.

Memory trick: T → M → S → D = Transform → Measure → Sort → Draw.

3. STEP-BY-STEP WORKING

STEP 1 — Transform

Objects exist in 3D world coordinates, but the camera has its own position and orientation. The polygons are converted into camera space so their positions can be evaluated relative to the camera.

Viva: First, the polygons are transformed into camera space so that their positions can be evaluated relative to the camera.

STEP 2 — Measure Depth

The project uses the centre of each polygon as the sorting reference. For a four-corner polygon, the corners are averaged to find its centre. The vector between the camera and that centre is then converted into a straight-line distance.

Formula: $d = \sqrt{(x^2 + y^2 + z^2)}$

Larger distance = farther polygon. Smaller distance = nearer polygon.

STEP 3 — Sort

Once distances are known, the polygons are sorted from farthest to nearest. For example, if a wall is 15 units away, a table is 8 units away and a monitor is 4 units away, the drawing order is Wall → Table → Monitor.

STEP 4 — Draw

The sorted polygons are drawn in order. Because each new polygon is painted over what is already on the screen, the nearer polygon covers the farther polygon wherever they overlap.

Key rule: FAR FIRST → NEAR LAST

4. UNDERSTANDING YOUR ACTUAL PROJECT CODE

You do not need to memorize every line of code. The important implementation parts are **face_depth()**, **painter_sort()** and the drawing loop.

face_depth(face, cam)

This function averages the four corners of a polygon to find its centre and returns the vector between the camera and that centre. The rendering process takes the length of that vector as the polygon's distance from the camera. That distance becomes the sorting key.

> **Viva answer:** face_depth finds the polygon centre and determines the camera-to-centre vector; its length is used as the polygon's distance for sorting.

painter_sort(items)

Each item contains the distance, face and lighting information. The function sorts by distance using reverse=True, so the farthest polygon is first and the nearest is last.

> **Viva answer:** painter_sort sorts the polygons according to their distance from the camera, from farthest to nearest.

Drawing loop

The rendering loop draws the sorted polygons one by one. Nearer surfaces are therefore painted after farther surfaces and cover them.

Camera.to_camera and to_screen

to_camera converts a world point into camera coordinates such as right, up and depth. to_screen performs perspective projection. Dividing by depth makes far objects appear smaller.

clip_near

It removes parts of a polygon that are behind or too close to the camera. Your project uses a near plane of 0.1 to avoid invalid perspective calculations.

5. IMPORTANT PROJECT DESIGN CHOICES

Why are walls split into 1.5 m tiles?

> A very large wall can contain parts at different depths. Sorting only by its centre can therefore be inaccurate. Splitting long walls into smaller 1.5 m tiles makes the centre a better approximation of each polygon's depth.

Why are floors drawn first?

Floors are always behind objects standing on them, so the project draws them first without sorting.

What is back-face culling?

Back-face culling skips polygon faces that point away from the camera. It saves drawing time and helps remove incorrect overlaps inside a box or room.

What is perspective projection?

Perspective projection makes distant objects appear smaller than nearby objects, helping the scene look like a 3D environment.

Why Python and Pygame?

Python is used for the logic and mathematical implementation. Pygame provides simple 2D polygon drawing, while the 3D mathematics and depth ordering are implemented in the project code.

6. FEATURES OF YOUR PROJECT

- 4 rooms + corridor
- Hinged doors that open when the player approaches
- Walls divided into 1.5 m tiles
- Back-face culling
- Near-plane clipping
- Distance fog
- Collision detection
- Live depth labels
- Minimap
- Depth shading

7. LIVE DEMONSTRATION CONTROLS

Key	Action
W / A / S / D or arrows	Move; A/D can be used for strafing
Q / E or mouse drag	Turn
P	Painter's Algorithm ON / OFF
G	Depth shading
L	Object depth labels
- / =	Replay draw order, far to near

Strongest demonstration: Press P to turn Painter's Algorithm off and show incorrect overlaps. Turn it on again and show the correct rendering. Press G to demonstrate depth shading.

8. LIMITATIONS AND Z-BUFFER

Painter's Algorithm limitations

- Cyclic overlap can make a complete drawing order impossible.
- Intersecting polygons can break the ordering assumption.
- Sorting costs O(n log n) for n polygons.
- Hidden polygons are still painted, causing overdraw.

What is cyclic overlap?

If A covers B, B covers C and C covers A, there is no single complete back-to-front order. This is called cyclic overlap.

Painter’s Algorithm vs Z-buffer

Painter’s Algorithm	Z-buffer
Sorts polygons	Stores depth per pixel
Farthest → nearest	Compares pixel depths
Requires polygon sorting	Does not require polygon sorting
Can cause overdraw	Handles visibility per pixel
Uses sorting time	Uses additional depth memory

9. COMPLETE VIVA QUESTIONS & ANSWERS

1. What is Computer Graphics?

Answer: Computer Graphics is the field of computer science that deals with creating, representing, manipulating and displaying images and visual objects using computers. Our project uses computer graphics concepts to create and render a 3D indoor environment.

2. What is the visibility problem?

Answer: The visibility problem is determining which objects or surfaces should be visible to the camera and which are hidden behind other objects.

3. What is Painter's Algorithm?

Answer: Painter's Algorithm is a visibility algorithm that renders polygons from the farthest to the nearest. Nearer polygons are drawn later, so they cover farther polygons.

4. Why is it called Painter's Algorithm?

Answer: It works like a painter: the background is painted first and foreground objects are painted later. Later paint covers earlier paint.

5. What are the other names?

Answer: Depth-Sorting Algorithm and Back-to-Front Rendering.

6. Why do we need depth ordering?

Answer: To correctly determine which surface is in front of another surface. Without it, a farther object may incorrectly appear over a nearer object.

7. What are the four steps?

Answer: Transform, Measure Depth, Sort and Draw. Memory trick: T $\rightarrow$ M $\rightarrow$ S $\rightarrow$ D.

8. What is camera space?

Answer: Camera space is a coordinate system where objects are represented relative to the camera. It helps determine their position and depth from the camera's point of view.

9. How do you calculate depth?

Answer: We find the polygon centre and calculate its straight-line distance from the camera using $d = \sqrt{(x^2 + y^2 + z^2)}$.

10. What is face_depth()?

Answer: It finds the polygon centre by averaging its four corners and determines the vector between the camera and the centre. Its length is used as the polygon distance.

11. What is painter_sort()?

Answer: It sorts polygons according to their distance from the camera, from farthest to nearest.

12. Why draw the farthest object first?

Answer: Because Painter's Algorithm follows a back-to-front approach. A nearer object drawn later will cover the farther object.

13. Why is the nearest object drawn last?

Answer: Because later drawing covers what was already drawn, so the nearer object appears in front.

14. Why are walls divided into 1.5 m tiles?

Answer: A very large wall may have different depths across its surface. Smaller tiles make the centre-based depth approximation more accurate.

15. What is back-face culling?

Answer: It skips polygon faces that face away from the camera because they cannot be seen. It saves drawing time and helps avoid incorrect overlaps.

16. What is near-plane clipping?

Answer: It removes parts of polygons that are too close to or behind the camera, preventing invalid perspective calculations.

17. Why Python?

Answer: Python is simple and suitable for implementing the mathematical logic and algorithm directly.

18. Why Pygame?

Answer: Pygame provides simple 2D polygon drawing, while the 3D mathematics and depth ordering are implemented in our own code.

19. What is a Z-buffer?

Answer: A Z-buffer stores a depth value for every pixel and compares depths to determine which object should be visible.

20. Painter's Algorithm vs Z-buffer?

Answer: Painter's Algorithm sorts complete polygons from farthest to nearest. A Z-buffer stores and compares depth for individual pixels.

21. What are the limitations?

Answer: Intersecting polygons, cyclic overlap, O(n log n) sorting cost and overdraw of hidden polygons.

22. What is cyclic overlap?

Answer: It occurs when A covers B, B covers C and C covers A, so there is no single complete back-to-front order.

23. What is the time complexity?

Answer: The sorting part is O(n log n) for n polygons. Drawing is approximately O(n) plus pixel-filling work.

24. What happens when P is pressed?

Answer: P switches the Painter's Algorithm on or off. OFF can show incorrect overlaps; ON restores correct depth ordering.

25. What happens when G is pressed?

Answer: G activates depth shading: near objects appear brighter and far objects appear darker.

26. How do you prove the algorithm works?

Answer: Press P to turn it off and show incorrect overlaps, then turn it on and show that nearer objects correctly cover farther objects. G can also demonstrate depth.

27. What is the final outcome?

Answer: A working 3D indoor navigation environment where overlapping surfaces are rendered correctly using depth ordering.

10. READY-TO-SPEAK PROJECT EXPLANATIONS

30-second answer: What is your project?

My project is 3D Indoor Navigation Using Depth Ordering. We implemented Painter's Algorithm, which is also called the Depth-Sorting or Back-to-Front Rendering algorithm. The main purpose is to correctly render overlapping surfaces. We calculate the distance of each polygon from the camera, sort the polygons from farthest to nearest, and draw them in that order. Since nearer polygons are drawn later, they cover farther polygons and produce the correct visual result.

1-minute answer: Explain how your algorithm works.

First, the polygons in the indoor environment are transformed into camera space. Then we find the centre of each polygon and calculate its straight-line distance from the camera using the 3D distance formula. After obtaining the distances, we sort the polygons from farthest to nearest. Finally, we draw them in that order. Because the nearer polygon is drawn later, it covers the farther polygon. This is the basic principle of Painter's Algorithm. In our project, we also use wall tiling, back-face culling and near-plane clipping to improve rendering.

11. LAST-MINUTE REVISION SHEET

ONE SENTENCE TO MEMORIZE

Painter’s Algorithm draws polygons from farthest to nearest, so nearer polygons are painted later and cover farther polygons.

THE COMPLETE CHAIN

Problem: Objects overlap.
Solution: Need depth ordering.
Algorithm: Painter’s Algorithm.
Rule: FAR → NEAR.
Process: Transform → Measure → Sort → Draw.
Result: Near objects cover far objects.
Limitation: Intersecting/cyclic polygons.
Alternative: Z-buffer.

MUST-KNOW TERMS

Visibility problem • Depth ordering • Camera space • Polygon • Face centre • Depth • Perspective projection • Painter’s Algorithm • Depth sorting • Back-to-front rendering • Back-face culling • Near-plane clipping • Z-buffer • Overdraw • Cyclic overlap

MUST-KNOW FORMULA

$d = \sqrt{(x^2 + y^2 + z^2)}$

MUST-KNOW COMPLEXITY

Sorting = O(n log n)

MUST-KNOW FUNCTIONS

face_depth() → determines polygon depth/distance reference.
painter_sort() → sorts polygons farthest to nearest.
Drawing loop → draws polygons in sorted order.

DEMO KEYS

P = Painter ON/OFF | G = Depth shading | L = Depth labels | - / = Replay draw order

Study tip: Don’t memorize only the answers. Understand the sequence Far → Near → Draw. If you understand that, most viva questions become easy.

Source basis: your Painter’s Algorithm poster/project material and viva preparation guide supplied in this chat.